

SECTION -A

Q1. Justify your constraint choices for at least three columns across two tables. What data integrity problems would arise if these constraints were not applied?

Ans: For a food delivery database, I would use the following constraints:

- restaurant_id — **PRIMARY KEY:**
It uniquely identifies each restaurant and prevents duplicate IDs.
- name — **NOT NULL:**
Every restaurant should have a name. It prevents records from being created without a restaurant name.
- restaurant_id in menu_items — **FOREIGN KEY:**
It connects each menu item to a valid restaurant. It prevents menu items from referring to a restaurant that does not exist.
- item_name — **NOT NULL:**
Every menu item should have a name.

Q2. Explain why the WHERE clause is essential in each DML statement. What could go wrong with an UPDATE or DELETE executed without a proper WHERE condition?\

Ans: The WHERE clause is important because it specifies **which records should be updated or deleted**.

To update only pending orders older than 7 days

```
UPDATE orders SET status = 'Cancelled' WHERE status = 'Pending' AND  
order_date < CURRENT_DATE - INTERVAL 7 DAY;
```

For updating unavailable menu items from closed restaurants

```
DELETE FROM menu_items WHERE status = 'Unavailable' AND  
restaurant_id IN (
```

```
    SELECT restaurant_id FROM restaurants WHERE status = 'Closed');
```

Without a proper WHERE condition:

- An UPDATE could change **every row** in the table.
- A DELETE could permanently remove **every row** in the table.

Q3. Which SQL clauses and aggregate functions would you use to produce this report? Explain why HAVING must be used here rather than WHERE to filter by aggregate results.

Ans: For the revenue dashboard, I would use:

- **SUM()** — calculates total revenue.
- **AVG()** — calculates average order value.
- **COUNT()** — counts the number of orders.
- **GROUP BY** — groups orders by restaurant.
- **HAVING** — filters restaurants based on the aggregate result.

```
SELECT restaurant_id, SUM(total_amount) AS total_revenue,  
AVG(total_amount) AS average_order_value, COUNT(*) AS order_count  
FROM orders WHERE MONTH(order_date) = MONTH(CURRENT_DATE)  
AND YEAR(order_date) = YEAR(CURRENT_DATE) GROUP BY  
restaurant_id HAVING COUNT(*) > 20;
```

Q4. Compare INNER JOIN and LEFT JOIN for this query. Which would you use and why? Describe what each join type does to records that have no match in the other table.

Ans: 1. An INNER JOIN returns only records where a matching restaurant exists.

```
SELECT o.order_id, r.name AS restaurant_name, o.total_amount  
FROM orders o INNER JOIN restaurants r ON o.restaurant_id = r.restaurant_id;
```

2. A LEFT JOIN returns **all orders**, even when there is no matching restaurant.

```
SELECT o.order_id, r.name AS restaurant_name, o.total_amount  
FROM orders o LEFT JOIN restaurants r ON o.restaurant_id = r.restaurant_id;
```

LEFT JOIN should be used because the question requires every order to appear, including orders that reference a restaurant no longer present in the table.

Q5. Justify why a VIEW is preferred over direct table access here. What are two limitations the front-end developer should be aware of when working with this VIEW?

Ans: A **VIEW** is preferred because it provides a simplified and controlled way for the front-end team to access required data without directly accessing all the underlying tables.

```
CREATE VIEW top_restaurants AS SELECT r.name, COUNT(o.order_id) AS  
total_orders, AVG(r.rating) AS average_rating FROM restaurants r LEFT JOIN  
orders o ON r.restaurant_id = o.restaurant_id GROUP BY r.restaurant_id,  
r.name;
```

Advantages

1. It hides unnecessary raw ID columns from the front-end team.
2. It simplifies complex queries.
3. It provides controlled access to selected data.

Two limitations

1. The front-end developer can only access the columns included in the VIEW.
2. A VIEW containing aggregate functions such as COUNT() and AVG() generally cannot be directly updated like a simple table.

Q6. Which database mechanism should have prevented this inconsistency? Explain how COMMIT, ROLLBACK, and SAVEPOINT work together to ensure both operations succeed or neither takes effect.

Ans: The problem occurs because the wallet deduction and payment insertion are not being treated as one transaction.

A **transaction** should be used so that either both operations succeed or both are cancelled.

COMMIT

COMMIT permanently saves all changes made during the transaction.

ROLLBACK

ROLLBACK cancels the changes made during the transaction.

SAVEPOINT

SAVEPOINT creates a point inside a transaction to which we can roll back if necessary.